DS 3010 Homework 8

Instructions: Homework is to be submitted on Canvas by the deadline stated above. Please clearly print your name and student ID number on your HW.

Show your work (including calculations) to receive full credit. Please work hard to make your submission as readable as you possibly can — this means no raw R output or code in your solutions (unless it is asked for specifically or needed for clarity).

Code should be submitted with your homework as a separate file (for example, a `.R` file, text file, Word file, or `.Rmd` are all acceptable). You should mark sections of the code that correspond to different homework problems using comments (e.g. `##### Problem 1 #####`).

Problem 1: Concept Review

(a) Explain in plain language (using limited statistics terminology) why lasso can set some of the regression coefficients to be 0 exactly, while ridge regression cannot. You may include a figure if that is helpful.

Solution: We did not discuss this in a huge amount of detail, so it would have been sufficient for students to say something like: the shape of the L1 penalty allows some of the regression coefficients to be set to 0, while the shape of the L2 penalty results in regression coefficients that are shrunk towards zero (but cannot be zero exactly).

(b) Suppose we estimate the regression coefficients in a linear regression model by minimizing

$$\sum_{i=1}^{n}\left(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij}\right)^2 + \lambda\sum_{j=1}^{p}\beta_j^2$$

for a particular value of λ. For parts (a) through (e), indicate which of i. through v. is correct. Justify your answer.

a. As we increase λ from 0, the training MSE will:

   i. Increase initially, and then eventually start decreasing in an inverted U shape.
   ii. Decrease initially, and then eventually start increasing in a U shape.
   iii. Steadily increase.
   iv. Steadily decrease.
   v. Remain constant.

   Solution: (iv). As λ increases, the amount of bias in the model increases and the model becomes less flexible (more constrained). Therefore, the training RSS will steadily increase as λ increases from 0.

b. Repeat (a) for test MSE.

   Solution: (ii) The test RSS will decrease initially (when the variance decrease offsets the bias) but eventually will start increasing in a U shape when the increase in bias can no longer offset the variance.

c. Repeat (a) for variance.

Solution: (iv) As we increase λ, the model becomes less flexible (more constrained) which will result in the variance will steadily decreasing.

d. Repeat (a) for (squared) bias.

Solution: (iii). As λ increases, the amount of bias in the model increases.

e. Repeat (a) for irreducible error.

Solution: (v) The irreducible error is unaffected by the choice of λ. It is the random noise that exists in our data and cannot be modified by our model.

Problem 2: Regularized Regression

For this problem, we will use the `College` data set in the `ISLR2` R package. Our aim is to predict the number of applications (`Apps`) received using the other variables in the dataset.

(a) Split the data set into a training and a test set. Please `set.seed(12)` so that we can all have the same results.

Solution:

```
library(ISLR2)
set.seed(12)
n = dim(College)[1]
x = model.matrix(Apps~.,data=College)[,-1]
Y = College$Apps
train_index = sample(1:n,n/2,replace=FALSE)
```

(b) Fit a ridge regression model (using all predictors) on the training set. The function `glmnet`, by default, internally scales the predictor variables so that they will have standard deviation 1. Explain why this scaling is necessary when implementing regularized models.

Solution: Consider that our estimates of β^R depend not only on the value of λ, but also on the scaling of the jth predictor. If our predictors are on different scales, then the predictors will have different contributions to the penalty term (for example, suppose you have predictors such as Age versus Income). To avoid this and apply the penalty evenly, we need to scale all our predictors so that they have standard deviation 1.

```
library(glmnet)
grid = 10^seq(10,-2,length=100)
ridge.College = glmnet(x[train_index,],Y[train_index],alpha=0, lambda=grid)
```

(c) Find an optimal λ for the ridge regression model on the training set by using 5-fold cross-validation. Report the optimal λ here.

Solution: The optimal λ is 4.641589.

```
cv.out = cv.glmnet(x[train_index,],Y[train_index],alpha = 0, lambda = grid, nfolds=5)
bestlambda = cv.out$lambda.min
```

(d) What is the value of the ℓ_2 norm of the estimated regression coefficients (excluding the intercept) associated with the optimal λ for ridge regression? You can evaluate this from the training set.

Solution: The l2 norm of the estimated regression coefficients associated with the optimal λ fro ridge regression is 539.2139. If students forgot the square-root, it?s ok, please still give them full credit.

```
coef_lambda_min = coef(cv.out, s = "lambda.min")
l2 = sqrt(sum(coef_lambda_min[2:18]^2))
```

(e) Using that optimal λ, evaluate your trained ridge regression model on the test set. Report the test MSE obtained.

Solution: The test MSE is 1431803.

```
ridge_prediction = predict(ridge.College,s = bestlambda,newx = x[-train_index,])
test_MSE = mean((ridge_prediction - Y[-train_index])^2)
```

(f) Find an optimal λ for the lasso regression model on the training set by using 5-fold cross-validation. Report the optimal λ here.

Solution: The optimal λ for the lasso regression model is 14.17474.

```
lasso.College = glmnet(x[train_index,], Y[train_index],alpha=1, lambda=grid)
cv.College<- cv.glmnet(x[train_index,],Y[train_index], alpha = 1,
lambda = grid, nfolds = 5)
bestlambda <- cv.College$lambda.min
bestlambda
```

(g) What is the value of the ℓ_1 norm of the estimated regression coefficients (excluding the intercept) associated with the optimal λ for lasso regression? You can evaluate this from the training set.

Solution: The l1 norm of the estimated regression coefficients associated with the optimal λ for lasso regression is 609.6691.

```
coef_lambda_min = coef(cv.College, s = "lambda.min")
l1 = sum(abs(coef_lambda_min[2:18]))
```

(h) Using that optimal λ, evaluate your trained lasso regression model on the test set. Report the test MSE obtained.

Solution: The test MSE is 1484667.

(i) Comment on your results. How accurately can we predict the number of college applications received? Is there much difference among the test errors resulting from these two approaches?

Solution: The test MSEs for both methods are quite close, 1431803 for ridge regression model and 1484667 for lasso regression model - both exceed 1,400,000 which in terms of the magnitude of Apps is not terribly accurate. Ridge regression model has slightly more accurate prediction compared to lasso regression model for this problem.

Problem 3: Tuning λ

For this problem, we will continue with the `Hitters` example from lecture. Our aim is to predict the salary of baseball players based on their career statistics.

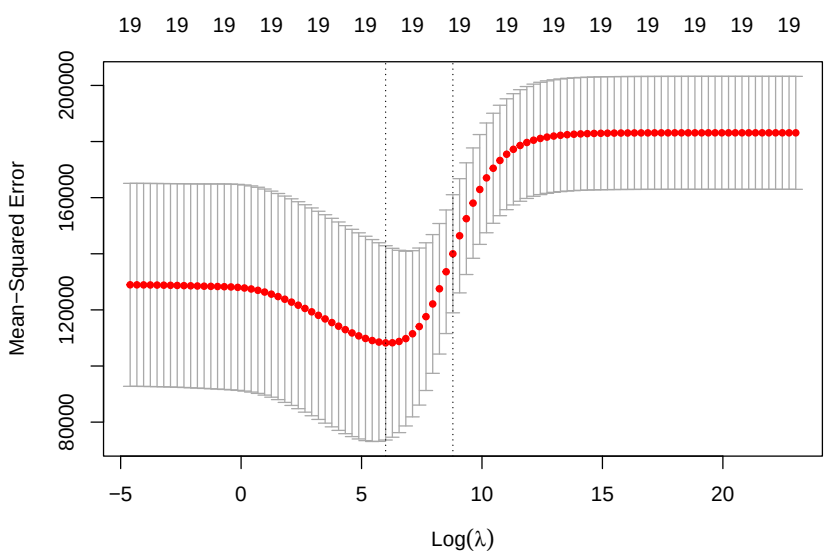
(a) We will start with a little data cleaning. We?ll also split the data into a training and test set. So that we all get the same results, please use the following code:

```
library(ISLR2)
Hitters = na.omit(Hitters)
n = nrow(Hitters) #there are 263 observations
x = model.matrix(Salary ~.,data=Hitters)[,-1] #19 predictors
Y = Hitters$Salary
set.seed(1)
train = sample(1:nrow(x), nrow(x)/2)
test=(-train)
Y.test = Y[test]
```

(b) Fit a ridge regression model. Replicate the example we had in class to obtain the the optimal λ that minimizes the 10-fold CV. Present a plot of the cross-validation error as a function of λ. Report that value here and call it $\lambda_{\text{min}}^{\text{ridge}}$.

Solution: $\lambda_{\text{min}}^{ridge} = 403.7017$

```
lambdagrid = 10^seq(10,-2,length=100)
cv.out = cv.glmnet(x[train,],Y[train],alpha=0, lambda=lambdagrid)
plot(cv.out)
lambda_min = cv.out$lambda.min
```



(c) Naturally, if we had taken a different training/test set or a different set of folds to carry out cross-validation, our optimal λ and therefore test error would change. An alternative is to select λ using the one-standard error rule. The idea is, instead of picking the λ that produces the smallest CV error, we pick the model whose CV error is within one standard error of the lowest point on the curve you produced in part (b). The intention is to produce a more parimonious model. The `glmnet` function does all of this hard work for you and we can extract the λ based on this rule using the following code: `cv.out$lambda.1se` (assuming your `cv.glmnet` object is named `cv.out`). Report your that λ here and call it $\lambda_{1se}^{\text{ridge}}$.

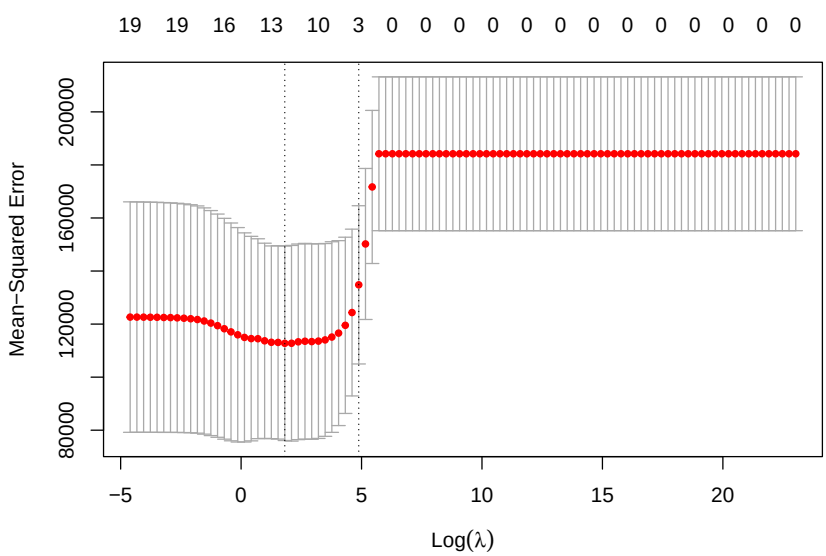
Solution: $\lambda_{1se}^{ridge} = 6579.332$.

```
lambda_1se = cv.out$lambda.1se
```

(d) Fit a lasso regression model. Replicate the example we had in class to obtain the the optimal λ that minimizes the 10-fold CV. Present a plot of the cross-validation error as a function of λ. Report that value here and call it $\lambda_{\text{min}}^{\text{lasso}}$.

Solution: $\lambda_{\text{min}}^{lasso} = 6.135907$.

```
cv.out.lasso = cv.glmnet(x[train,],Y[train],alpha=1, lambda=lambdagrid)
plot(cv.out.lasso)
lambda_lasso_min = cv.out.lasso$lambda.min
```



(e) For lasso, report the optimal λ using the smallest standard error rule and called it $\lambda_{1se}^{\text{lasso}}$.

Solution: $\lambda_{1se}^{lasso} = 132.1941$.

```
lambda_lasso_1se = cv.out.lasso$lambda.1se
```

(f) Evaluate the ridge regression models on your test set using $\lambda = \lambda_{\min}^{\text{ridge}}$ and $\lambda = \lambda_{1se}^{\text{ridge}}$. Evaluate the lasso models on your test set using $\lambda_{\min}^{\text{lasso}}$ and $\lambda_{1se}^{\text{lasso}}$. Report the test MSEs from these 4 models.

Solution: The test MSE for ridge was 139020.3 and 165523.3, using $\lambda_{\min}^{ridge}$ and λ_{1se}^{ridge}, respectively. The test MSE for lasso was 144217.6 and 157773.7, using $\lambda_{\min}^{lasso}$ and λ_{1se}^{lasso}, respectively.

```
### test MSE
ridge.train = glmnet(x[train,],Y[train],alpha=0,lambda=lambdagrid)
lasso.train = glmnet(x[train,],Y[train],alpha=1,lambda=lambdagrid,)
ridge.pred = predict(ridge.train,s = lambda_min,newx=x[test,])
mean((ridge.pred-Y.test)^2)
ridge.pred2 = predict(ridge.train,s = lambda_1se,newx=x[test,])
mean((ridge.pred2-Y.test)^2)
lasso.pred1 = predict(lasso.train,s = lambda_lasso_min,newx=x[test,])
mean((lasso.pred1-Y.test)^2)
lasso.pred2 = predict(lasso.train,s = lambda_lasso_1se,newx=x[test,])
mean((lasso.pred2-Y.test)^2)
```

(g) Report the coefficient estimates from ridge using $\lambda_{\min}^{\text{ridge}}$ and $\lambda_{1se}^{\text{ridge}}$ and likewise for the lasso models. How do the ridge regression estimates compare to those from the lasso? How do the coefficient estimates from using $\lambda_{\min}$ compare to those from the one-standard error rule? Discuss what you observe.

Solution: Students should create a table comparing the estimated coefficients. They should observe that lasso can set the regression coefficients to 0 while ridge cannot. The larger the penalty, the smaller the coefficients. So for example, $\lambda_{1se}^{lasso} = 132.1941$ is larger than $\lambda_{\min}^{lasso} = 6.135907$, which results in more aggressive shrinking towards 0. We can see that more coefficients are set to 0 for lasso with λ_{1se}^{lasso} compared to $\lambda_{\min}^{lasso}$. Similar observations can be made for ridge regression.

```
### coefficients refit on the full dataset
lasso_min.final = glmnet(x,Y,alpha=1,lambda = lambda_lasso_min)
lasso_1se.final = glmnet(x,Y,alpha=1,lambda = lambda_lasso_1se)
ridge_min.final = glmnet(x,Y,alpha=0,lambda = lambda_min)
ridge_1se.final = glmnet(x,Y,alpha=0,lambda = lambda_1se)
results = cbind(coef(ridge_min.final), coef(ridge_1se.final),
coef(lasso_min.final), coef(lasso_1se.final))
colnames(results) = c('ridge.min','ridge.1se','lasso.min','lasso.1se')
```

	ridge.min	ridge.1se	lasso.min	lasso.1se
(Intercept)	22.14199384	342.454584686	41.8141746	316.02183375
AtBat	0.09202637	0.055333136	-0.6006949	.
Hits	0.79612092	0.211556250	3.5685719	0.72112214
HmRun	0.80036765	0.769972808	.	.
Runs	1.03423625	0.349986914	.	.
RBI	0.87793980	0.359997480	.	.
Walks	1.54008233	0.442625783	3.0277367	0.56907852
Years	1.82022134	1.618528505	-4.0742639	.
CAtBat	0.01134633	0.004665257	.	.
CHits	0.05434354	0.017500453	.	.
CHmRun	0.38690022	0.131097056	0.1608128	.
CRuns	0.10821779	0.035107946	0.3585243	0.09666601
CRBI	0.11395518	0.036259312	0.4081870	0.25371287
CWalks	0.06088065	0.036996438	-0.1445588	.
LeagueN	19.66599802	0.609025156	26.5127706	.
DivisionW	-72.32536945	-10.288271440	-118.3247590	.
PutOuts	0.15296151	0.026487928	0.2476295	.
Assists	0.02456701	0.004086746	.	.
Errors	-1.15478574	-0.042674529	-0.5820076	.
NewLeagueN	9.44765833	0.820596873	.	.

(h) Which model performs the best in terms of prediction? Explain any intuition as to why.

Solution: Ridge regression with $\lambda_{\min}^{ridge}$ gave us the smallest test MSE (for this particular training and test set). These results tell us that using ridge regression with a smaller penalty lead to improved results. This suggests that perhaps all the predictors in the model are somewhat important in prediction (hence the choice of ridge over lasso and a smaller penalty).

(i) If you were to make a recommendation to an upcoming baseball player who wants to make it big in the major leagues, what handful of features would you tell this player to focus on?

Solution: I would tell the player to focus on getting as many hits, walks, homeruns, and RBIs as possible.